SOFTWARE REQUIREMENTS SPECIFICATION

RENTAMOVIL

Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana

Ramírez Vargas

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

Updated version — October 28, 2026

1. INTRODUCTION

The RentaMovil project consists of developing a web and mobile platform to optimize the rental of private vehicles. Its goal is to streamline the reservation process, reduce wait times, and improve the control, tracking and security of the fleet through geolocation tools, notifications and online management. This SRS document defines the scope, objectives and functional and non-functional requirements of the system, serving as a guide to ensure that the final product meets the needs of the users and of the company.

1.1 Title: RentaMovil

Rental and control of private vehicles.

1.2 Problem or need

There are problems related to congestion in the reservation process and long wait times for renting vehicles at physical locations. In addition, there is poor management of the control, tracking and security of rented vehicle fleets.

How can a technological solution be implemented that optimizes the reservation process, reduces wait times, and improves the control, tracking and security of rented vehicle fleets?

1.3 Justification

The growth of the vehicle rental market has exposed deficiencies in traditional management, especially at physical locations, where manual reservations and long wait times affect the user experience. According to Statista (2024), this industry will reach $120 billion by 2026, which demands greater operational efficiency. McKinsey studies (2023) show that digitalization can reduce wait times by up to 60% and improve efficiency by 35%. In addition, the International Transport Forum highlights that technology improves fleet control and security. For this reason, it is essential to implement a comprehensive technological solution that optimizes reservations and strengthens vehicle tracking.

1.4 General objective

Design and implement a comprehensive technological system that optimizes the vehicle reservation process and strengthens the control, tracking and security of rented vehicle fleets.

1.5 Specific objectives

1. Analyze the main deficiencies in the current reservation and fleet management process at vehicle rental companies.
2. Design a technological platform that allows efficient, real-time online reservations.
3. Implement geolocation and tracking tools to improve the control and security of rented vehicles, limited to vehicles currently on rent.
4. Add administrative features for the complete management of the respective vehicle fleets, including their maintenance cycle.
5. Integrate in-app notification features for relevant reservation and payment events.
6. Evaluate the impact of the proposed system on reducing wait times and improving operational security.

1.6 Scope

This project focuses on the design, development and implementation of a comprehensive technological solution for managing reservations and controlling vehicle fleets at car rental companies. The system will target both customers and administrators of the service.

Includes:

- Digitalization of the reservation process through a web and mobile platform.

- Customer registration with email verification by code, login, and password recovery via a temporary code.
- Customer profile management: name, phone (optional), profile photo and password change from the active session; changing the email requires confirming with the current password.
- Optional insurance selection and acceptance of terms and conditions during the reservation.
- Payment by bank transfer: the customer scans a QR code for one of the company's bank accounts, transfers the money and uploads the receipt; an Administrator reviews it and approves or rejects it (there is no integration with an automated payment gateway — verification is manual).
- Execution of the rental (pickup/return) with invoice generation.
- Geolocation for real-time tracking, limited to vehicles currently on rent.
- Fleet status and availability module, with an inventory that administrators can filter by status.
- Management of the vehicle maintenance lifecycle (scheduled, in progress, completed or cancelled), with a history per vehicle.
- Branch management, including its location on a map (latitude/longitude).
- In-app notifications for customers and administrators.

Does not include:

- Driver's license validation.
- Integration with an automated payment gateway (card/PSE processor); payment verification remains manual, done by an Administrator, not an external provider call.
- A mobile app for administrators (web only).
- A full maintenance work-order module with external workshops or quotes — the system records the maintenance lifecycle (scheduled/in progress/completed/cancelled), cost and notes, but not a third-party quoting flow.
- The concept of a "driver" distinct from the customer/reservation holder.
- Cancellation of reservations by the customer: a reservation only becomes CANCELLED when an Administrator definitively rejects the payment or when it automatically expires after 24 hours; reservation dates cannot be modified either (only the return branch).

1.7 Personnel Involved

Name	Role	Professional category	Responsibility	Contact
Nicole Dayana Ramírez Vargas	Lead	Trainee, Technologist in Software Analysis and Development	Lead, plan and oversee the project delivery	nicolerv18007@gmail.com
David Santiago Erez Rodríguez	Developer	Trainee, Technologist in Software Analysis and Development	Build the web application	erezsantiago25@gmail.com
Thiago Fabian Rojas Guevara	Analyst	Trainee, Technologist in Software Analysis and Development	Information analysis, design and programming	thiagorojasg451@gmail.com

1.8 Definitions, acronyms and abbreviations

Term	Description
User	A person who will use the application (Customer, Administrator or Super Admin)
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
SENA	Servicio Nacional de Aprendizaje (Colombia's national vocational training service)
HU	User Story, the format each requirement was captured in before being written up as an FR

1.9 References

Document title	Reference
Standard IEEE 830-1998	IEEE
The team's internal technical documentation (user stories, domain model, API contracts)	rtm-docs repository

1.10 Summary

The RentaMovil project arises as a response to the current deficiencies in the vehicle rental sector, specifically in manual reservation processes and fleet control. Through this system, customers will be able to make reservations online, track vehicles in real time during the rental, and have every paid reservation invoiced automatically. With a focus on efficiency, security and customer experience, RentaMovil aims to become a key tool for modernizing vehicle rental management.

2. GENERAL DESCRIPTION

2.1 Product Perspective

RentaMovil is a comprehensive technological solution designed to digitize and optimize the private vehicle rental process. The backend is implemented as five microservices in Java + Spring Boot with Hexagonal Architecture — Identity & Access (rtm-iam), Fleet & Maintenance (rtm-fleet-maintenance), Booking & Reservation (rtm-booking-reservation, which also hosts rental execution and notifications), Payment & Billing (rtm-payment-billing) and Telemetry & GPS (rtm-telemetry-gps) — exposed through an API Gateway. All of them use a single SQL Server instance with one schema and one user per service, migrated with Liquibase (one migrations repository per service), and each service records its own audit trail. The frontend is delivered as a web application (React + Vite) and a mobile application (Expo + React Native), both for customers; administrative management is done exclusively through the web. File storage (payment receipts, QR codes, profile photos, vehicle photos and maintenance photos) is done in Cloudinary.

2.2 User Characteristics

Role	Background	Activities
Customer (CLIENT)	General public	Browses the catalog, selects a vehicle and insurance, creates and views their reservations, pays, views their invoices, manages their profile
Administrator (ADMIN)	Business administration or related	Manages the fleet, branches, maintenance, reservations, payments and billing for the whole company; registers pickups/returns; checks GPS location
Super Admin (SUPER_ADMIN)	Business administration or related	Everything the Administrator can do, plus managing administrative users (granting/revoking roles, unblocking accounts) and bank accounts

Design note: an Administrator or Super Admin can also hold a personal reservation (using their own account as a customer) — the role determines what they can administer, not who can hold a reservation.

2.3 Constraints

- 2.3.1. The application requires an internet connection to function.
- 2.3.2. Users must have the latest version of the mobile application to ensure compatibility and security.
- 2.3.3. The software must be compatible with mobile devices (Android/iOS) and modern web browsers.
- 2.3.4. Personal data protection regulations must be met, since sensitive user and vehicle information is handled.

- 2.3.5. The accuracy of the geolocation system depends on the quality of the device's GPS and the available network.
- 2.3.6. The backend must be implemented in Java + Spring Boot with Hexagonal Architecture, on a shared SQL Server instance (one schema per service), migrated with Liquibase.
- 2.3.7. Payment is processed via bank transfer with manual review: the customer uploads a receipt and an Administrator approves or rejects it — there is no integration with an automated payment gateway.
- 2.3.8. GPS tracking only applies to vehicles in RENTED status (currently rented out).

2.4 Assumptions and Dependencies

- 2.4.1. It is assumed that the described requirements are stable and accepted by the stakeholders.
- 2.4.2. The devices on which the application runs must meet the established technical requirements.
- 2.4.3. Reservations and notifications are handled in real time, as long as there is a stable connection.
- 2.4.4. The system's success will depend on proper adoption and interaction by users (customers and administrators).
- 2.4.5. The system must be able to scale and allow new vehicle units to be registered as demand grows.
- 2.4.6. Each user has exactly one role (CLIENT, ADMIN or SUPER_ADMIN) — there are no multi-role users.

3. SPECIFIC REQUIREMENTS

3.1 Common Interface Requirements

3.1.1 User Interface (Customer)

- Browsing and filtering available vehicles.
- Selecting a vehicle and an optional insurance plan.
- Creating and viewing their own reservations, with return-branch change.
- Paying for reservations by bank transfer (receipt upload).
- Viewing and downloading invoices.
- Profile management (name, phone, photo, email, password).
- Receiving in-app notifications.

3.1.2 User Interface (Administrator / Super Admin)

- Vehicle management (registration, editing, deactivation).
- Branch management, including its location on a map.
- Fleet maintenance lifecycle management (schedule, start, complete, cancel).
- Managing reservations for the whole company.
- Registering vehicle pickup/return.
- Viewing payments and billing.
- GPS tracking and route history.
- Managing administrative users and bank accounts (Super Admin only).

3.1.3 Hardware Interfaces

- Stable internet connection.
- A recent version of the software and operating system.
- At least 500 MB of free storage for the mobile application.

3.1.4 Software Interfaces

- Operating System (PC): Windows 7 or later, macOS 11 or later, modern Linux distributions.
- Operating System (Mobile): Android 10 or later, iOS/iPadOS 15 or later.
- Browser: Chrome, Firefox, Edge, Safari and equivalent modern browsers.

- Backend: Java + Spring Boot, SQL Server, Liquibase.

3.1.5 Communication Interfaces

Communication between clients, servers and external services is done over secure HTTPS and TLS 1.2+ protocols. Internal communication between microservices uses HTTP with a shared internal API key (X-Internal-Key header) on /internal/** routes, which the API Gateway does not expose; GPS devices authenticate with a device API key. There is no message broker: events between services travel over internal HTTP.

3.2 Functional Requirements — Summary

ID	Module (Bounded Context)	Description	Responsible service
FR1	Identity & Access	Customer registration with email verification, authentication and account management	rtm-iam
FR2	Fleet & Maintenance	Public vehicle catalog and availability lookup	rtm-fleet-maintenance
FR3	Booking & Reservation	Creating, viewing and modifying (return branch) own reservations	rtm-booking-reservation
FR4	Rental Execution	Registering the physical pickup and return of the vehicle	rtm-booking-reservation (rental module)
FR5	Payment & Billing	Payment by transfer with receipt review, reservation confirmation, own invoicing	rtm-payment-billing
FR6	Telemetry & GPS	Location tracking and route history	rtm-telemetry-gps
FR7	Notification	In-app notifications	rtm-booking-reservation (notification module)
FR8	Administration (cross-cutting)	Administrative management of users, fleet, maintenance, branches, reservations, insurance and payments — see the "Domain" field on each item	various (see domain per item)

3.3 Functional Requirements Description

FR 1.1 — Customer registration

Field	Value
Identifier	FR 1.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Associated UI: registration form (first name, last name, email, username, optional phone, password), validation and confirmation messages.

Input: the visitor fills in first name, last name, email, username and password, and optionally a phone number, and submits the form.

Output: the Person and the User are created (role CLIENT) in PENDING_VERIFICATION status and a 6-digit code valid for 15 minutes is emailed; the account becomes active once it is verified.

Description: the system lets an unauthenticated visitor register so they can create reservations. The username is chosen by the visitor and must be unique; the phone number is optional — if it is not provided, the person is left without a registered phone number.

Abnormal situation handling: if the email is already registered, the system rejects the registration with the corresponding message; if the username is already taken, the system rejects it with a different message, without creating a duplicate account.

Acceptance criteria:

- The form requires first name, last name, email, username and password; the phone number is optional.
- The email and the username must be unique in the system.
- The created account automatically receives the CLIENT role.
- An account in PENDING_VERIFICATION cannot log in until it confirms the 6-digit code sent to its email (valid for 15 minutes; it can be resent).

FR 1.2 — Login

Field	Value
Identifier	FR 1.2
Type	Functional Requirement
Critical?	Yes
Priority	High

Associated UI: login form (email/username + password).

Input: the user enters their credentials.

Output: an access token (30 min) and a refresh token (7 days) if the credentials are valid.

Description: the system authenticates the user and issues the corresponding JWT tokens.

Abnormal situation handling: if the credentials are invalid, the system responds with 401 without indicating whether the email or username exists (to avoid leaking information). After 5 consecutive failed attempts against the same account, the account moves to BLOCKED status and rejects login (423) even with the correct password; only a Super Admin can unlock it. The attempt counter is kept in the service memory and is reset by a successful login. In addition, the API Gateway rate-limits sensitive routes (login, registration and password recovery) to 10 requests per IP every 5 minutes (responds 429). An unverified account only gets the email-not-verified notice if the password is correct; an INACTIVE account cannot log in.

Acceptance criteria:

- Only users with status = ACTIVE can log in.
- The access token expires after 30 minutes; the refresh token after 7 days, rotating on every use (a refresh token is used only once).
- On the fifth consecutive failed attempt the account becomes BLOCKED (responds 423) until a Super Admin unlocks it.
- The API Gateway limits login, registration and password recovery to 10 requests per IP every 5 minutes (responds 429).

FR 1.3 — Password recovery

Field	Value
Identifier	FR 1.3
Type	Functional Requirement
Critical?	Yes
Priority	High

Associated UI: recovery request form, new password form.

Input: the user requests recovery with their email; they receive a 6-digit code valid for 15 minutes and enter it together with the new password.

Output: the password is updated; the code is marked as used; all of the user's active sessions are closed.

Description: the system allows regaining access through a temporary code sent to the registered email. The request always responds generically (202), whether or not the email exists, to avoid revealing which emails are registered in the system.

Abnormal situation handling: if the code is expired, was already used, or does not match the last one issued for that account, the system rejects the operation without updating the password.

Acceptance criteria:

- The code has 6 digits and expires after 15 minutes; only the last code issued is valid.
- A used code cannot be reused.
- Completing the recovery closes all of the user's active sessions.

FR 1.4 — Account information management

Field	Value
Identifier	FR 1.4
Type	Functional Requirement
Critical?	Yes
Priority	Medium

Input: the authenticated customer updates their name, phone and/or profile photo from their profile; or, to change their email, they enter the new email together with their current password.

Output: the Person's data is updated; if it was an email change, the new value is reflected in the profile only after the password is confirmed.

Description: the system keeps the customer's personal information up to date. The profile photo is a URL already uploaded to Cloudinary from the frontend; the system only accepts URLs from the project's Cloudinary account. The username cannot be changed once the account is created.

Abnormal situation handling: if the password entered to change the email does not match, the system rejects the change and the email stays the same; if the new email is already registered by another Person, it is rejected with the corresponding message; a photo whose URL does not belong to the project's Cloudinary account is rejected.

Acceptance criteria:

- Name, phone and photo are updated without additional confirmation.
- Changing the email requires re-entering the current password and only applies if it matches.
- The username remains immutable from this screen.

FR 1.5 — Password change (active session)

Field	Value
Identifier	FR 1.5
Type	Functional Requirement
Critical?	Yes
Priority	Medium

Input: the authenticated customer enters their current password and the new password.

Output: the password is updated; all of the user's sessions are closed, including the current one.

Description: lets an already-authenticated user change their password from their profile, without going through the code-based recovery flow (FR1.3), reusing the current password as identity confirmation.

Abnormal situation handling: if the current password does not match, the system rejects it; if the new password is the same as the current one, it is also rejected.

Acceptance criteria:

- The new password must follow the same rule as registration (at least 8 characters, at least one uppercase letter and one number).
- After the change, the user must log in again on all of their devices.

FR 2.1 — Browse and filter the vehicle catalog

Field	Value
Identifier	FR 2.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: a visitor (with or without a session) accesses the catalog and optionally applies filters (category, brand, engine, price, dates).

Output: a list of vehicles with brand, model, category, daily price and image; or an empty state if there are no matches.

Description: the system lets anyone browse the public vehicle catalog without needing to authenticate. Without an explicit status filter, the catalog only shows AVAILABLE vehicles. The same endpoint, with an Administrator session, supports an additional status filter for administrative use (see FR8.2).

Abnormal situation handling: if there are no AVAILABLE vehicles, or the filter returns no results, an empty state is shown without an error. Combining the date filter with a status other than AVAILABLE is rejected, because the date filter only makes sense on the public catalog.

Acceptance criteria:

- The catalog is accessible without a session.
- Filters can be combined (category + brand + engine + price).

FR 2.2 — Check a vehicle's availability

Field	Value
Identifier	FR 2.2
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the customer checks a vehicle's availability for a date range.

Output: confirmation of availability or not, based on the vehicle's status and the existing reservations that overlap with the range.

Description: the system computes availability by combining Vehicle.status and the vehicle's active reservations — it is not a stored field, it is computed on the fly.

Acceptance criteria:

- A vehicle with a reservation that overlaps the requested range is marked as unavailable.
- An AVAILABLE vehicle with no overlaps is marked as available.

FR 3.1 — Create a reservation (vehicle, insurance, dates and branches)

Field	Value
Identifier	FR 3.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the authenticated customer selects an available vehicle, optionally an insurance plan (First Aid, Standard or All-Risk), pickup/return dates, pickup and return branches (which may differ at no extra cost), and accepts the terms and conditions.

Output: a Reservation is created in PENDING_PAYMENT status; event ReservationCreated.

Description: the system creates the reservation, computing vehicle_subtotal, insurance_subtotal (if applicable) and total_amount. The term "driver" does not exist in this model — the reservation holder (client_id) is always the authenticated Person who creates it.

Abnormal situation handling: if end_date ≤ start_date, or the customer did not accept the terms and conditions, the reservation is rejected.

Acceptance criteria:

- A reservation has exactly one vehicle, which cannot be changed once created.
- At most one insurance plan per reservation, charged only once.
- Pickup and return branches may differ, with no extra charge.
- It is not possible to continue without accepting the terms and conditions.

FR 3.2 — View and modify own reservations

Field	Value
Identifier	FR 3.2
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the customer views their reservations and, if applicable changes the return branch.

Output: a list of their own reservations with their status; or an updated reservation with the ReservationReturnBranchChanged event, depending on the change.

Description: a reservation cannot be cancelled by the customer or have its dates changed; it can only become CANCELLED when an Administrator definitively rejects the payment or when it automatically expires after 24 hours (FR5.1). The only thing the customer can modify is the return branch.

Abnormal situation handling: changing the return branch with less than 3 days left before end_date is rejected.

Acceptance criteria:

- A customer can have multiple active reservations at the same time.
- The return branch can only be changed up to 3 days before end_date; it cannot be changed on a CANCELLED or COMPLETED reservation.
- The customer has no operation to cancel a reservation; cancellation only happens through definitive payment rejection or expiration (FR5.1).

FR 4.1 — Register vehicle pickup

Field	Value
Identifier	FR 4.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the Administrator, at the branch, registers the pickup of a vehicle for a CONFIRMED reservation, with the initial mileage and the assigned GPS.

Output: a Rental is created in IN_PROGRESS; Vehicle.status = RENTED; event RentalStarted.

Abnormal situation handling: a reservation that is not CONFIRMED cannot start its Rental.

Acceptance criteria:

- The Rental is only created from a CONFIRMED Reservation.

FR 4.2 — Register vehicle return

Field	Value
Identifier	FR 4.2
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the Administrator registers the return with the final mileage.

Output: Rental.status = COMPLETED; Vehicle.status = AVAILABLE; Reservation.status = COMPLETED; event RentalCompleted.

Abnormal situation handling: a final_mileage lower than initial_mileage is rejected.

Acceptance criteria:

- final_mileage must always be ≥ initial_mileage.

FR 5.1 — Pay for a reservation by bank transfer

Field	Value
Identifier	FR 5.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the customer selects a BankAccount (one of the company's bank accounts, shown as a QR code), transfers the exact total_amount of their reservation outside the system, and uploads the payment receipt. An Administrator later reviews that receipt and approves or rejects it.

Output: a Payment is created in PENDING_REVIEW; Reservation.status = PENDING_REVIEW; event PaymentSubmitted. After the Adminis trator's review: if approved, Payment.status = APPROVED, Reservation.status = CONFIRMED, event PaymentCompleted; if rejected, Payment.status = REJECTED, event PaymentRejected, and Reservation.status goes back to PENDING_PAYMENT (if the Administrator allows a retry) or moves to CANCELLED (if the Administrator rejects it without allowing a retry).

Description: the payment is a real bank transfer, manually verified by an Administrator — there is no integration with an automated payment gateway (card/PSE processor) in this MVP. The receipt and the QR code are stored in Cloudinary.

Abnormal situation handling: if the amount does not exactly match total_amount, the system rejects the receipt upload before creating the Payment — there are no partial payments or "outstanding balance." If no receipt is approved within 24 hours of reservation_date, the reservation automatically expires to CANCELLED.

Acceptance criteria:

- The amount declared by the customer must exactly equal the reservation's total.
- A rejected receipt does not confirm the reservation; the Administrator decides whether it goes back to PENDING_PAYMENT (retry) or rejects it definitively (the reservation becomes CANCELLED).
- Without an approved receipt within 24 hours, the reservation is cancelled automatically.

FR 5.2 — Generate and view an invoice

Field	Value
Identifier	FR 5.2
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: after a PaymentCompleted, the system automatically generates the invoice; the customer views and downloads it afterward.

Output: an Invoice with a consecutive invoice_number and a PDF generated once and stored; event InvoiceGenerated.

Description: the generated document is a billing invoice, not a digitally signed legal contract.

Abnormal situation handling: a second invoice is never generated for the same reservation even if the event is reprocessed (idempotency by reservation_id).

Acceptance criteria:

- The invoice is only generated if there was a successful payment.
- The PDF served on every download is always the same file (it is not regenerated).

FR 6.1 — Track the location of a rented vehicle

Field	Value
Identifier	FR 6.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: the Administrator checks the location of a rented vehicle.

Output: the last recorded Location (latitude/longitude).

Description: the GPS device assigned to the rental reports its position to the telemetry service, authenticating with a device API key. Only reports from devices linked to an IN_PROGRESS rental are accepted; reports with out-of-range coordinates or at (0,0), which is interpreted as "no GPS fix", are rejected. The Administrator queries the last position through the same service.

Abnormal situation handling: a customer (role = CLIENT) who tries to access this endpoint gets a 403.

Acceptance criteria:

- Tracking only happens while Vehicle.status = RENTED.
- Only ADMIN/SUPER_ADMIN can check location.

FR 6.2 — View route history

Field	Value
Identifier	FR 6.2
Type	Functional Requirement
Critical?	Yes
Priority	Medium

Input: the Administrator views the complete route of an already-finished rental.

Output: a set of Location points in chronological order.

Abnormal situation handling: if there are no recorded points (GPS failure), an empty state with an informational note is shown, with no error.

Acceptance criteria:

- The route corresponds exactly to the IN_PROGRESS period of the queried Rental.

FR 7.1 — In-app notifications

Field	Value
Identifier	FR 7.1
Type	Functional Requirement
Critical?	Yes
Priority	High

Input: domain events (ReservationCreated, PaymentSubmitted, PaymentCompleted, PaymentRejected, ReservationExpired, ReservationReturnBranchChanged, MaintenanceCompleted, etc.).

Output: a Notification is created for the corresponding recipient (customer or Administrator).

Description: the system notifies users inside the application about relevant reservation and payment events, with real-time delivery on the web (SSE) and push notifications on the mobile app; it does not use SMS, and email is only used for verification and password-recovery codes.

Acceptance criteria:

- Every relevant event generates a notification to the correct recipient (customer vs. Administrator).
- The recipient can mark a notification as read.

FR 8.1 — Administrative user management

Field	Value
Identifier	FR 8.1
Type	Functional Requirement
Domain	Identity & Access
Critical?	Yes
Priority	High

Input: a Super Admin selects an existing Person and assigns the ADMIN or SUPER_ADMIN role; they can also deactivate an ACTIVE account or reactivate it, and unblock a BLOCKED account (locked after 5 failed login attempts).

Output: user.role_id or user.status is updated; the RoleGranted event is emitted when applicable.

Description: the system lets the Super Admin delegate or revoke administrative access, including for other Super Admins, and control the status of any account.

Abnormal situation handling: if a user with the ADMIN role (not Super Admin) tries to change roles or statuses, the system responds with 403 — this operation is exclusive to SUPER_ADMIN.

Acceptance criteria:

- Only SUPER_ADMIN has permission to grant/revoke roles and change an account's status.
- Deactivating an account (INACTIVE) does not delete its reservation or payment history.
- Every role or status change is audited.

FR 8.2 — Manage vehicles (register, edit, deactivate, inventory)

Field	Value
Identifier	FR 8.2
Type	Functional Requirement
Domain	Fleet & Maintenance
Critical?	Yes
Priority	High

Input: the Administrator registers a new vehicle (plate, model, category, engine, daily price, branch) or edits/removes an existing one; they can also list the full inventory, filtering by status (AVAILABLE, RENTED, MAINTENANCE, RETIRED or ALL).

Output: a vehicle created in AVAILABLE status; or updated/removed; event VehicleRegistered when applicable; or a filtered inventory listing.

Description: the system lets the Administrator keep the fleet catalog up to date and view it as an internal inventory, unlike the public catalog in FR2.1 which only exposes AVAILABLE vehicles.

Abnormal situation handling: a duplicate plate is rejected; a vehicle with a non-cancelled future reservation, or in RENTED status, cannot be removed until it has no pending reservations and goes back to AVAILABLE.

Acceptance criteria:

- The plate is unique in the system.
- A vehicle with an active reservation cannot be removed.
- Filtering the inventory by status requires an Administrator session; without authentication, only the public AVAILABLE catalog is returned.

FR 8.3 — Send a vehicle to maintenance and complete it

Field	Value
Identifier	FR 8.3
Type	Functional Requirement
Domain	Fleet & Maintenance
Critical?	Yes
Priority	High

Input: the Administrator marks an AVAILABLE vehicle for maintenance; later, they mark that maintenance as finished.

Output: when sending it, Vehicle.status = MAINTENANCE and a VehicleMaintenance is created in IN_PROGRESS, with the VehicleSentToMaintenance event; when completing it, the VehicleMaintenance moves to COMPLETED with end_date, Vehicle.status goes back to AVAILABLE, and MaintenanceCompleted is emitted.

Description: the system lets the operational status of each vehicle be controlled and shows aggregated statistics by status. The details of scheduling, editing, cancelling and the maintenance history are described in FR8.7.

Abnormal situation handling: a RENTED vehicle cannot be sent to maintenance directly.

Acceptance criteria:

- The 4 valid Vehicle statuses are AVAILABLE, RENTED, MAINTENANCE, RETIRED. RETIRED is a logical deactivation — the vehicle is not physically deleted, it just stops appearing in the catalog and cannot be reserved again.
- The statistics panel shows the vehicle count by status.

FR 8.4 — Branch management

Field	Value
Identifier	FR 8.4
Type	Functional Requirement
Domain	Fleet & Maintenance
Critical?	Yes
Priority	Medium

Input: the Administrator creates, edits or removes a branch, and can place it on a map.

Output: the branch is available as a pickup/return option, with its latitude/longitude stored if it was placed on the map.

Description: the map location is optional — a branch created without coordinates is still valid, it just is not drawn on the application's maps.

Abnormal situation handling: a branch with assigned vehicles cannot be removed; a branch cannot be saved with only one of the two coordinates (latitude or longitude) present.

Acceptance criteria:

- A branch with associated vehicles cannot be removed until they are reassigned.
- Latitude and longitude are saved together or not at all.

FR 8.5 — Reservation and insurance administration

Field	Value
Identifier	FR 8.5
Type	Functional Requirement
Domain	Booking & Reservation
Critical?	Yes
Priority	Medium

Input: the Administrator views every reservation in the system (not only their own), and manages the available insurance plans (InsuranceType).

Output: a full list of reservations with filters; opening a reservation's detail also shows its payment information (amount, bank account used, uploaded receipt, and PENDING_REVIEW/APPROVED/REJECTED status); insurance plans created/edited.

Description: there is no separate "payment lookup" screen — the payment is shown as part of the detail of the reservation it belongs to, not as a separate module.

Acceptance criteria:

- The Administrator sees every reservation, including their own as a customer, with no special treatment.
- Each reservation's detail includes its associated payment information.
- A change to an insurance plan's daily_cost does not affect reservations already created.

FR 8.6 — Bank account management

Field	Value
Identifier	FR 8.6
Type	Functional Requirement
Domain	Payment & Billing
Critical?	Yes
Priority	High

Input: a Super Admin creates, edits or deactivates a BankAccount (bank, holder, QR code image).

Output: the bank account becomes available (or not) as a payment option at the customer's checkout.

Description: exclusive to SUPER_ADMIN — neither CLIENT nor ADMIN can manage the company's bank accounts, since this is sensitive financial information.

Abnormal situation handling: deactivating a bank account does not delete the payment history that already used it — it simply stops being offered as a new option.

Acceptance criteria:

- Only a user with the SUPER_ADMIN role can create, edit or deactivate bank accounts.

- The customer can only choose active accounts (is_active = true) when paying.

FR 8.7 — Schedule, edit, cancel and view maintenance history

Field	Value
Identifier	FR 8.7
Type	Functional Requirement
Domain	Fleet & Maintenance
Critical?	No
Priority	Medium

Input: the Administrator schedules a future maintenance for a vehicle that is not RETIRED; they can start it, cancel it (instead of removing it), or view the full history filtering by vehicle and status.

Output: a VehicleMaintenance created as SCHEDULED (does not change the vehicle's status); starting it moves it to IN_PROGRESS (see FR8.3); cancelling it moves it to CANCELLED and, if it was IN_PROGRESS, the vehicle goes back to AVAILABLE.

Description: a maintenance's lifecycle has 4 states: SCHEDULED → IN_PROGRESS → COMPLETED, with the possibility of moving to CANCELLED from SCHEDULED or IN_PROGRESS. A maintenance is never physically deleted — "deleting" it means cancelling it, the same as retiring a vehicle. A COMPLETED or CANCELLED maintenance becomes read-only: none of its fields (type, dates, cost, notes, photo) can be edited once closed.

Abnormal situation handling: a maintenance can only be started if the vehicle is AVAILABLE; maintenance cannot be scheduled for a RETIRED vehicle; a vehicle has at most one IN_PROGRESS maintenance at a time, although it may have several SCHEDULED ones; retiring a vehicle automatically cancels its SCHEDULED and IN_PROGRESS maintenances.

Acceptance criteria:

- A scheduled maintenance does not affect the vehicle's availability until it starts.
- A closed maintenance (COMPLETED/CANCELLED) cannot be edited.
- The history can be filtered by vehicle and by status.

3.4 Non-Functional Requirements

NFR 1 — Security

The system requires authentication with role-based access control (CLIENT/ADMIN/ SUPER_ADMIN). Personal data is protected according to applicable regulations. Access is controlled via JWT: the access token expires after 30 minutes and the refresh token after 7 days, with mandatory rotation on every use. An account is locked after 5 consecutive failed login attempts (a Super Admin unlocks it) and the API Gateway rate-limits sensitive routes (login, registration and password recovery) per IP.

Acceptance criteria: login required for protected operations · permissions correctly enforced per role · no refresh token can be reused after it rotates.

NFR 2 — Performance

Design target: the system must respond in under 2 seconds for 95% of operations. The GPS location update frequency is set by the device installed in the vehicle; the system records every valid report and shows the last position received.

Acceptance criteria: p95 < 2 s (target) · the last position reported by the GPS device is reflected when the location is queried.

NFR 3 — Usability

The interface must be intuitive, responsive (working on mobile and desktop devices) and in Spanish for administrative staff and customers.

Acceptance criteria: equivalent functionality on PC, tablet and mobile · user interface in Spanish, with no unnecessary technical text.

NFR 4 — Maintainability

The architecture must be modular (microservices + hexagonal), with error logs accessible to the technical team and basic documentation per service.

Acceptance criteria: logs available to the technical team · changes in one microservice must not require changes in others to keep working (low coupling).

NFR 5 — Integration

The system must expose API contracts (OpenAPI) for each microservice, and integrate the GPS device and Cloudinary as file storage (payment receipts, QR codes, profile photos, vehicle photos and maintenance photos).

Acceptance criteria: OpenAPI contracts documented per service · failures in an external service (GPS, Cloudinary) must not bring down the rest of the system.

NFR 6 — Availability

Target: the system must be available at least 99% of the time each month (once in production), with recovery from failures in under 10 minutes.

Acceptance criteria: uptime ≥ 99% · effective recovery from failures ≤ 10 min.

NFR 7 — Data consistency

All microservices use a single SQL Server instance, each with its own schema and user. Migrations are versioned with Liquibase, with one repository per service, and applied in order: iam, fleet-maintenance, telemetry-gps, booking-reservation and payment-billing.

Acceptance criteria: no schema migration is deployed without review by the services that depend on those tables · migrations are versioned with Liquibase and applied in the order above.